

Luca D'Altri

Small project for my 2nd year undergraduate course on algorithms and data structures. The program has 19 functions, described from page 2.

Overview

The objective of the project is to build a single-file program that:

- Builds/import valid sentences of max 10 words
- Generates their SHA256 code
- Compares all the sentence hashes amongst them, char by char
- Find the hashes with the highest number of common chars, with respect to their order

I followed two targets:

1. Build at least 10^6 rigorously valid sentences
2. Be as quick as possible doing the comparisons

Target 1:

To build the sentence, I have used 6 variables, namely x1-6, which are arrays of strings. The structure of the sentence that I used is the following:

- (1) Uppercase pronoun (not at the 3rd singular person)
- (2) Adverb
- (3) Verb (not at the 3rd singular person)
- (4) Object
- (5) Bonus part
- (6) End

With 6 variables is possible to easily generate up to 10^9 strictly valid sentences. If so, it is convenient to import 2 txt files: adverbs (x2), and [verbs](#) (x3). However, a billion of sentences exceeds by far the computing capabilities of my machine, and the time limit too. With a contemporary machine, it does make sense to work with a 10^7 pool.

I built the software with an old laptop with Windows 10, 8gb ram, Intel 6200U. After experimenting limited computing capabilities, I decided to cap the number of generated sentences to $\sim 2 \cdot 10^6$. This pool can grant ~ 300 hours of ininterrupt mining at a speed of $\sim 3 \cdot 10^{10}$ comparisons/24h. The requirement is ~ 5 gb of free ram.

The validity of the sentences is 100%. Furthermore, potential loss of computation power is avoided by removing all duplicates in the imported txt, using the `correction()` method.

Target 2:

As regard the execution speed of the bottleneck, a **375% speed** is achieved compared to a string1-string2 comparison method. This is obtained by decomposing the String[] SHA256 array into a char[][] matrix of characters. Another crucial but more obvious optimization is the inner-loop variable index following the outer-loop variable (200% speed overall the sentences pool). Further details on the *compareMatrix()* description. Furthermore, all the class variables are locally imported within the comparison method.

Code Structure

The program is divided in 20 sections: class variables, plus 19 methods. Data structure used are mainly Arrays (1D, 2D, and ArrayList – partially dynamic array). Arrays are the quickest data structure available for my algorithms. Those algorithms require speed in accessing data at all positions. The array fixed size is not a drawback, because to modify the array once created. However, in the preliminary execution stages, the dynamic array is used because grants flexibility in size terms.

00 - Class Variables

The creation of the sentences is designed on 6 variables, namely x1, x2, x3, x4, x5, x6. These variables are initialized as private static class variables. In this way, the variables are potentially accessible by all methods within the class, without having to declare them in the main method. One of the variables (x3) is imported from a txt file containing 1041 compatible verbs. It is available at this address:

<https://github.com/aaronbassett/Passphrase/blob/d5cc4ae01aa857bd89b38600c7207f9208278d1f/verbs.txt>. The only manual modification I apported to the file is the deletion of the verb "be".

01 - main()

The main method is minimal. It contains just calls to other methods, and the returned variables. The procedure is:

1. Provide instructions
2. Load the x3 variable (verbs not at the 3rd singlar person)
3. Remove duplicates from the imported list
4. Generate the array of sentences

5. Generate SHA256 hashes
6. Give info on sentences and comparisons pool size, plus alert if 8Gb ram are required
7. Convert the String[] array of hashes in char[][] matrix of hashes
8. Start dinamically mining from index i0 compared to i0+1, i0+2, ..., i0+(n-1). Then start the outer loop again with i1, i2, ..., i(n-1).
9. Print on screen the best 10 findings

02 - instructions()

The method provides a description of the program. Created to keep clean the main() method.

03 - load()

The original method was provided by lecturer. My custom load method repeatedly asks the user to input the file name of the text document to load, until a valid document is loaded. The check is done using regex. The second custom choice is to return ArrayList<String>, which is a dynamic array, ideal for a temporary storage of my imported words.

04 - readWords()

The method reads in the text document. Called by load(), the readWords method was provided by the lecturer. I added the closure of the buffer (br.close();), and a print statement.

05 - correction()

The method checks if there are any duplicates in the imported list. If so, the duplicates are deleted. After correction() runs, we are sure there are no duplicates:

1. Saving computation power
2. Avoiding the potential bug of 64 matching characters

The algorithm sorts the ArrayList and removes any subsequent equal strings. The complexity is $O(n)$, plus the pre-built Collections.sort().

06 - savedComputationPower()

Calculates the difference of the number of sentences generated before and after the *correction()*, and the subsequent number of comparisons avoided. The

difference is converted in percentage, then approximated to 1 decimal place. The comparisons are calculated by a loop with increasing index: the 1st hash is compared with all the others (n-1), the 2nd with n-2, because the comparison between 1st and 2nd has been already performed with the 1st loop. Eg: for n=10 sentences, we have 45 comparisons (10-1 + 9-1 + ... + 1-1). Complexity: $O(n)$.

07 - reduceNumber()

savedComputationPower() requires converting into <double> a division between <long> data types. If the <long> has more than 6 digits, then it is trimmed. Then the division between the new <double>s is performed, converted, and rounded in *savedComputationPower()*. Complexity: $O(1)$.

08 - sentenceGenerator()

Generates valid sentences through 6th level nested loops, starting from the outer loop taking care of x1, and ending with the last inner loop doing x6. The sentence is built in the 6th level inner-loop, concatenating 6 variables at 6 specific indexes.

The sentences are stored in a `String[]` array, which is returned.

Theoretical complexity: $O(n^6)$. Real complexity: $O(k1*k2*n*k3*k4*k5)$, where k are constants = $O(n)$.

09 - sha256()

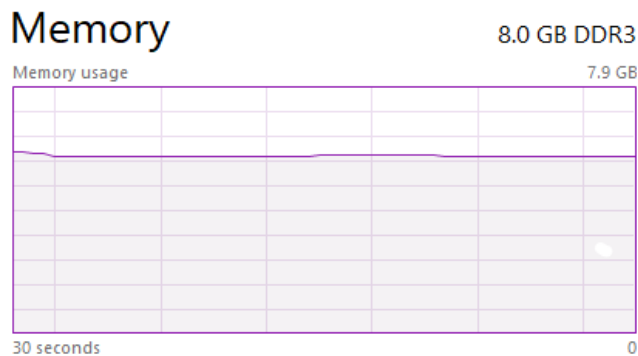
Given method: it generates the SHA256 hash of a `String`. Complexity: $O(1)$.

10 - convert()

Convert the array of sentences into an array of SHA256 hashes. The algorithm uses a for loop and a call to the *sha256()* method. A new `String[]` is returned. Complexity: $O(n)$.

11 - info()

Gives info on the amount of valid sentences generated, plus the amount of total comparison needed to exhaust the pool. The total amount of comparison is obtained with a for loop that tracks the sum of comparison needed for every index of the array of sentences: starting from 0 comparisons at last index, then adding +1 comparison for the penultimate, and so on until the first index is reached (length-1 comparisons this time). If the estimate ram needs exceed 4gb, an alert message is printed to console. With a 2.3 million sentences, the software requires approx 5Gb of free ram on Windows 10. Complexity: $O(n)$.



12 - check() - for strings comparison (not used)

Import 2 strings and check for the same character at the same index. The final count of matches is returned. To avoid useless comparisons, the loop breaks at 53/64 if the current count ≤ 10 . In this way I **avoided 15% of comparisons**, obtaining 2% of extra speed on my machine.

13 - compare() - for strings comparison (not used)

Import the array of sentences and their sha256 correspondent strings. The method compares by for loop the index $<i>$ with its subsequent $<i+1>=j$. An $<int>$ array is returned, storing matching count and sentence indexes.

14 - charMatrix()

The original $\text{String}[\text{length}]$ of SHA256 hashes is converted into a $\text{char}[\text{length}][64]$ matrix. Complexity: $O(n*64) = O(n)$.

15 - compareMatrix() - 375% speed

The method takes in the $\text{char}[][]$ matrix (and the $\text{String}[]$ of sentences, just to print in real time the 20+ findings and their index).

The method prints live any any match ≥ 20 chars. This is a way to track the current index without losing computation power or printing too many statements.

The method does not need to call any other method (it just very rarely calls the findingsManager() method for 20+ matches findings, with no performance impact).

The comparison is made amongst subsequent rows of the matrix: $\text{row}[i]$ and $\text{row}[i+1]$.

Using a 3rd level nested loop, the algorithm keeps track of the column, which is the index of the character in the original String.

By using the matrix, all info is **pre-loaded** in a 2D map (the matrix itself), that stores both value and index. Therefore, Java does not need to access 2 strings every time, trillions of times.

Time in milliseconds - outer loop - Java 17.0 - Win10, 6200U, 8gb ram

String[] method()

```
MINING .....  
1728  
1613  
1620  
1615  
1796  
1625  
1930  
1673  
1635  
1611  
1609  
1612  
1612  
1610  
1605  
1606  
1681
```

char[][] method()

```
MINING .....  
473  
449  
447  
437  
435  
438  
437  
439  
435  
439  
434  
437  
437  
439  
437  
436  
433
```

Avg = $1650/440 = 3.75 = 375\%$ of the original speed

The best 10 matches are automatically returned at execution-end. However, the findings ≥ 20 are printed in real time, to allow tracking the index. Complexity: $O(n \cdot (n/2) \cdot 64) = O(n^2)$. The bottleneck of the program.

This method does not include the try-catch structure. After monitoring the results, the try-catch structure slows the machine by ~ 15 milliseconds for every cycle.

16 - findingsManager()

Manages in real time the best 10 matches in an ordered list. It uses a single loop of a bubble-sort algorithm. There is no need of multiple loops because the matrix starts from an ordered status: the emptiness, from which I start adding the best findings. Complexity: $O(k)$, $k=10 = O(1)$.

17 - printResult()

The best matching strings are printed to console when the pool has been exhausted. I decided to store and print the best 10 to have the opportunity to compare them. Complexity: $O(k)$, $k=10 = O(1)$.

18, 19 - elapsedStart(), elapsedEnd()

Monitor the elapsed time within the program, converted in millisencond for better readability. Fundamental to verify speed improvements. The elapsed time is relative to the machine in use: the results slightly/considerably change with a different processor, amount of ram, OS, JVM version. However, this remains an absolutely valid way to monitor the relative performance. Complexity: $O(1)$.

Appendix

Verb list: <https://github.com/aaronbassett/Pass-phrase/blob/master/verbs.txt>